TOC Module 03

Main

(250)

(Numaricals Exam Pov)

All Sums are available at YT Search for learning & finishing this Module in 5h

Q1. Construct an Context free Grammer for

1. any number of a's over a set $\Sigma = a$

$L = \{\epsilon, a, aa, aaa \ldots\}$

$RE = a^*$

Production $P \to S \to aS/\epsilon$

$G = (V\,T\,P\,S)$

$G = (\{S\}, \{a\}, \{S \to aS/\epsilon\}, S)$

2. Any Number of a's and b's

$L = \{\epsilon, a, b, ab, ba, aa, bb \ldots\}$

$RE = (a+b)^*$

$G = (\{S\}, \{a, b\}, \{S \to aS/bS/\epsilon\}, S)$

II. Construct a Context free Grammer for the following language

3. language Containing atleast 2 a's

$L = \{aa, aab, abaab \ldots\}$

$RE = (a+b)^* a (a+b)^* a (a+b)^*$

$S = Aa\,Ab\,A$

$S \to Aa\,Aa\,\epsilon$

$\to \epsilon a \epsilon a b \epsilon \to$ 'aab'

$G = \{\{S, A\}, \{a, b\} \{S \to AaAaA, A \to Aa / Ab / \epsilon\}, S)$

(4) One Instance of 000

Σ 0,1

$L = \{000, 10001, ---\}$

$RE = (0+1)^* 000 (0+1)^*$

$S \to A000A$

$A \to A0 / A1 / \epsilon$

0001 , 1

$S \to A000A$

$S \to A00A1$

$\to 0001$

10000

$S \to A\ 000\ A$

$\to 1A000A0$

$\to 10000$

$G = (\{S A\}, \{0, 1\}, \{S \to A000A$
$A \to A0 / A1 / \epsilon\}, \{S\})$

(5) Set of strings with Equal a's & b's

$\Sigma = a, b$

$L = \{ab, aabb, abab ---\}$

$S \to aSbS / \epsilon$

$S \to bSaS / \epsilon$

(abab)

$S \to aSbS$

$\to abaSbS$

$\to abab$

$G = \{\{S\}, \{a, b\}, \{S \to aSbS$
$S \to bSaS / \epsilon\}$

(6) $RE = (0+1)^*$ Write the Context free grammer

Σ 0,1

$L = \{\epsilon, 0, 1, 00, 10, 11 ---\}$

$G = 0S / 1S / \epsilon$ $\quad G = (\{S\}, \{0, 1\}, \{S \to 0S / 1S / \epsilon\}, \{S\})$

) Construct CGF for the following to accept panlendrom over ab

Σab

$L = \{aa, bb, aba, bab, aabaa, abba - \}$

$S \to aSa$

$S \to bSb / a / b / \epsilon$

$G = (\{S\} \{ab\} \{S \to aSa, S \to bSb/a/b/\epsilon\} \{S\})$

(8) Construct an CFG for $L = \{a^n b^n \} n \geq 1 \}$

$L = \{aa, bb, abab - - \}$

→ $S \to aSb$

$S \to aS / bS / \epsilon$

'aaabbb'

$S \to aSb$

$S \to a\,aSb$

$S \to aa\,bSb$

$S \to aaa\,bSb$

$S \to aaa\,bbSb$

$S \to aaa\,bbb$

$G = (VTPS)$

$G = (S, ab, S \to aSb, S \to aS/bS/\epsilon, S)$

(9) $\{a^n b^n \mid n \geq 0, n > 0\}$

$L = \{\epsilon ab, aa, bb, aaa\,bbb - - \}$

$S \to AB$

$S \to aA / B$

$S \to bB / \epsilon$

$G = (VTPS)$

$G = [\{S, A, B\} \{a, b\} \{S \to AB, A \to aS/\epsilon, B \to bS/\epsilon\}, S\}$

(10) $\{w \mid n_a(w) = n_b(w)\}$

So, SS is for Mixing

<u>ww^R U</u>

So, $\rightarrow$ asb | bsa | ss / ϵ

$G = (VTPS) = (S, ab, S \rightarrow asb|bsa|SS/\epsilon), S)$

(11) $a^m b^m c^n, m, n > 0$

So, $S \rightarrow S_1 C$

$S_1 \rightarrow aS_1 b | \lambda$

$C \rightarrow cC$

(12) $\{a^n b^{2n} \mid n \geq 0\}$

$L = \{abb, aabbb, ---\}$

$S \rightarrow \{asbb / \epsilon\}$

$S \rightarrow AB$

$A \rightarrow aA/\epsilon$

$B \rightarrow bbB/\epsilon$

$G = (VTPS)$

$G = (\{S\}, \{a, b\}, \{S \rightarrow aSbb / \epsilon\}, S)$

(13) $L = \{a^{2n} b^n \mid n \geq 0, n > 0)$

$L = \{aab, aaabb ---\}$

$S \rightarrow aaSb/\epsilon$

$G = VTPS$

$G = (\{S\}, \{a, b\} \{S \rightarrow aaSb/\epsilon\}, \{S\})$

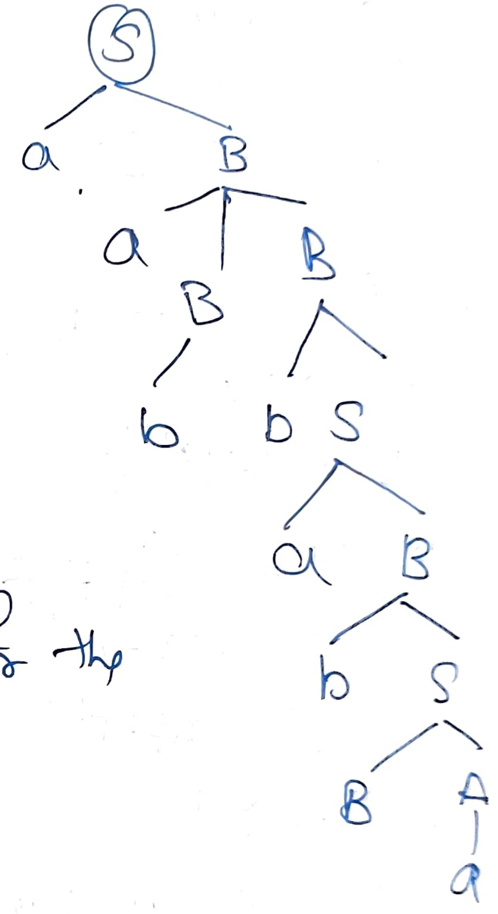
Derive a string aabbabba using LHD and Post free following.

$S \to aB \mid bA$

$A \to a \mid aS \mid bAA$

$B \to b \mid bS \mid aBB$

$S \to aB$
$S \to aaBB$
$S \to aabA$
$S \to aabbS$
$S \to aabbaB$
$S \to aabbabS$
$S \to aabbabbS$
$S \to aabbabba$



(15) Eleminate the left recursion for the following

$E \to E*T / T$

$E' \to *TE' / \epsilon$

$E \to TE'$

(16) $S \to S0S1S / 01$

$S \to 01S'$

$S' \to 0S1SS' / \epsilon$

(17) $A \to Aa/b$

$A \to bA'$

$A' \to aA'/\epsilon$

(18) $L \to LS/S$

$L \to SL'$

$L' \to SL'/\epsilon$

(19) $A \to Ab/b$

$A \to bA'$

$A' \to A'b/\epsilon$

(20) $A \to ABd \,/\, Aa \,/\, a$

$B \to Be \,/\, b$

$A \to aA'$

$A' \to \epsilon \,/\, BdA' \,/\, aA'$

$B \to Be/b$

$B \to bB'$

$B' \to \epsilon / eB'$

(21) (i) $E \to E + T / T$

$E \to TE'$

$E' \to +TE' / \epsilon$

(ii) $T \to T + F / F$

$T \to FT'$

$T' \to +FT'$

(iii) $F \to id$

$F \to id$ (No change No Reccursion of left)

(22) Remove left Reccursion

(i) $A \to Ba / Aa / c$

$A \to BA'$

$A' \to aA' / aA' / \epsilon$

(ii) $B \to Bb / Ab / d$

$B \to dB'$

$B' \to bB' / bB' / \epsilon$

23 - $S \to XYX$

$X \to 0X/\epsilon$

$Y \to 1Y/\epsilon$

} Null Production

$X \to \epsilon$

$Y \to \epsilon$

} identification of Null Production

To eleminiate $x \to \epsilon$ Replace Each occurance of X with ϵ

$S \to \bar{X}Y\bar{X} / Y\bar{X} / \bar{X}Y / \cancel{Y}$

To $X \to 0X/\epsilon$

$\to 0\epsilon \to 0$

$Y \to 1Y/\epsilon$

$\to XYX / YX / XY / Y$ {substitute $Y = \epsilon$}

$\to XX / X / 0$

$X \to 0X/0$

$Y \to 1Y/1$

24 - Unit Production

(i) $S \to 0A/1B/C$

$A \to 0S/00$

$B \to 1/A$

$C \to 01$

LHS & RHS Should be in form $A \to B$ for unit Production

$S \to C$

$B \to A$

} unit Productions

Elimination

$S \to C$

$C \to 01 \Rightarrow S \to 01$

$B \to A$

$A \to 0S/00 \Rightarrow B \to 0S/00$

$S \to 0A/1B/01$

$A \to 0S/00$

$B \to 1/0S/00$

$C \to 01$

(ii) $S \to AB$
$A \to a$
$B \to C/b$
$C \to D$
$D \to E/bc$
$E \to d/Ab$

unit Production

$B \to C$
$C \to D$
$D \to E$
} Remove unit Productions

$\hookrightarrow$ Eliminat $D \to E$
$E \to d/AB$
$\Rightarrow \boxed{D \to d/AB/bc}$

$\hookrightarrow$ Eleminate $C \to D$
$D \to E \Rightarrow C \to \boxed{d/AB}$

$\hookrightarrow B \to C,\ C \to D,\ B \to D$
$\Downarrow$
$\boxed{d/Ab/bc/b}$

final Grammer
$S \to AB$
$A \to a$
$B \to d/Ab/bc/b$
$C \to d/Ab$
$D \to d/Ab/bc$
$E \to d/Ab$

$\to$ Closer Property of CFL

(i) Union

$L_1 \to G_1$
$L_2 \to G_2$

$S_1 \to aS_1b/\lambda$
$S_2 \to aS_2b/\lambda$
$S \to S_1/S_2$

Closed under union

(ii) Concatenation $(\cdot)$

$S_1 \to aS_1b/\lambda$
$S_2 \to aS_2b/\lambda$
$S \to S_1 \cdot S_2$

iii) Clean closure (*)

$L_1 \to G_1$

$L_2 \to G_2$

$L_1^* \to G_1 \to S_1$

$\lambda / S S_1 \to S$

→ Intersection (X Not closure)

$L_1 \cap L_2$

If L_1 & L_2 are CFL then $L_1 \cap L_2$ is not CFL

Ex- $L_1 \{a^n \mid n \geq 0\}$ $L_1 = \{\epsilon, a, aa \cdots\}$

$L_2 \{b^n \mid n \geq 0\} = \{\epsilon, b, bb, bbb \cdots$

$L_1 \cap L_2 = \{\epsilon\}$

Not closed under Intersection

→ Con Catenation

if L_1 is CFL then Complement of L_1 is not a CFL

$L_1 = \{a^n \mid n \geq 0\}$ $L_1 = \{\epsilon, aa, aaa \cdots\}$

$L_2 = \{b^n / n \geq 0\}$ $L_2 = \{\epsilon, bb, bbb \cdots\}$

Hence not closed under Complement.

25 → CNF (COMSKY NORMAL FORM)

A Context free grammer is Said to be in CNF when

$A \to BC$

$A \to a$

The following is Conversion of
CFG (context free grammar) to CNF (Comsky Normal form)

Ex

$S \rightarrow bA \mid aB$
$A \rightarrow bAA \mid aS \mid a$
$B \rightarrow aBB \mid bS \mid b$

} No ϵ-Productio[n]
No Null Productio[n]
No Unit Productio[n]

So, $A \rightarrow a$
$B \rightarrow b$
} chomsky Normal form [CNF]

① $S \rightarrow \underline{bA} \Rightarrow C_bA$
$C_b \rightarrow b$

② $S \rightarrow aB \Rightarrow C_aB$
$C_a \rightarrow a$

$\cancel{A \rightarrow bAA}$
$\Rightarrow C_bAA$
$C_b \rightarrow b$

$A \rightarrow aS \Rightarrow C_aS$
$C_a \rightarrow a$

$B \rightarrow aBB$
$B \rightarrow C_aBB$
$C_a \rightarrow a$

$B \rightarrow bS \Rightarrow C_bS$
$C_b \rightarrow b$

$A \rightarrow C_bAA$
$A \rightarrow C_bD_1$
$D_1 \rightarrow AA$

$B \rightarrow C_aBB$
$B \rightarrow C_aD_2$
$D_2 \rightarrow BB$

$S \rightarrow C_bA$
$C_b \rightarrow b$
$S \rightarrow C_aB$
$C_a \rightarrow a$
$A \rightarrow C_bD_1$
$D_1 \rightarrow AA$
$A \rightarrow C_aS$

$B \rightarrow C_aD_2$
$D_2 \rightarrow BB$
$B \rightarrow C_bS$

} All these are in Chomsky Normal form

26- Decision Property of Context free language/Grammer

- → Emptyness Problem
- → Finitiness Problem
- → Membearship Problem
- → Equivance Proplem (No Solution) (There is no Produces) → (Undesidable problem)

Emptyness Problem, Finitiness Problem, Membearship Problem: decidable Problem

→ Emptyness Problem (Check if Can Produce Empty language or not)

(*) Check if Start Symbol has Useless symbols
(*) Remove Useless Symbols
(*) If Starting Symbol also has useless Symbol than Empty language else Non-Empty language

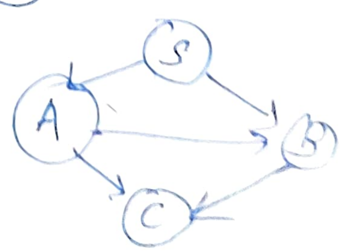
$S \to XY$
$X \to AA|AX$
$Y \to b$
$A \to a$

No usless Symbols
This is non-Empty language

→ Finitiness Problem check if the following Makes finite language or not.

(*) Simplify grammer
(*) check for CNF
(*) Construct CNF
(*) if graph has any loop then its infinite language

EX- SAB
$A \to Bc/a$
$B \to cc/b$
$C \to a$



Membership Problem

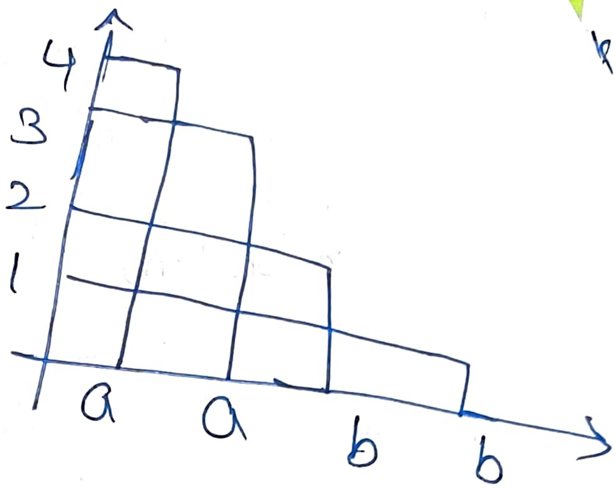
(*) Check wheather the given String is the Member of CNF or not

Ex - $S \rightarrow AB \mid BB$

$A \rightarrow CC \mid AB \mid a$

$B \rightarrow BB \mid CA \mid b$

$C \rightarrow BB \mid AA \mid b$



Q - baabc

	s	4	3	2	1
1	ASC	ϕ	ϕ	AS	B
2	ASC	B	B	AC	
3	B	S,C	A,C		
4	AS	B			
5	A,C				

If (1,5) has starting symbol then string is a Member

1 2 3 4 5
b a a b c

$12 \rightarrow (11)\cdot(22)$
$= B\cdot(AC)$
$= BA, BC$
$= A, S$

$13 = (11)(23)$
$= BB$ ✗

↳ $(12)(33)$
$(AS)(AC)$
$= AA, AC, SA, SC$
✗ ✗ ✗ ✗

$(4,5) = (44)(55)$
$= B\cdot(AC)$
$= BA, BC$
$= A \quad S$

$(2,3) = (22)(33)$
$= (AC)(AC)$
$= AA, AC, CA, CC$
✗ ✗ ✗ ✓

$(3,4) = (33)(44)$
$= (AC)(B)$
$= AB, CB$
$= S, C,$ ✗

$(24) = (2,3,4)$
Two ways $[2,34]$
or $[23,4]$

$24 = (22)(34)$
AS, AC, CS, CC
✗ ✗ ✗ B

$(23)(44) = BB$
✗

(35) = (345)

= (33)(45) or (34)(55)

= AC, AS = (SC) AC)

= AA, AS, CA, CS = SA, SC, CA, CC

X X X X X X X B

(14) = (1,2,3,4)

= (1,1)(2,3-4) or (12)(34) or (123)(4)

= (B)(24) = (AS)(SC) (13)(44)

= (B)(B) = AS, AC, SS, SC = ϕ

X X X X X X

(25) = (22)(35) or (23)(45) or (24)(55)

(AC)(B) (B)(AS) (B)(AC)

= AB, CB = BA BS (BA)(BC)

SC X = A X A S

(15) = (11)(25) or (12)(345) = ASC

= (B)(ASC) = (12)(35) or (123)(45)

(BA)(BS)(BC) = (AS)(B) = (13)(45)

A X S = AB, SB ϕ

= AS SC X

= SC

or (1234)(5)

(1,5) = ASC = (14)(55)

= ϕ

Module 03

(CNF) Context free Grammer

Theory

CAP → Non-Terminals (B)
Small → Terminals (a)

A Grammer (G) is 4 tuples G = VTPS where V is Set of Variables or non-Terminals
t → terminals
P → Set of Producers - Each Producer are in form of $\alpha \rightarrow \beta$ where α is a String $(V \cup T)^+$
$\beta \in (V \cup T)^*$
S → is the Starting Symbol

→ **Types of Grammer**
Type 0 (UnRestricted Grammer)
Type 1 (Context Sensestive)
Type 2 (Context free Grammer)
Type 3 (Regular Expression)

Notations Used While Constructing Grammer
The following are Terminals
(a) They key words such as if, for, while, dowhile etc
(b) digits from 0-9
(c) Symbols Such As + − * / etc
(d) The lower Case such As ID
(e) id

→ The following are Non-Terminals

(a) The lower cases names as Expressions, band, operator etc.
(b) Capital letters such As ABC etc
(c) The capital letters S is the Starting Symbol

→ UnRestricted Grammer (Type 0)

A Grammer (G) = (VTPS) is Said to be Type 0 Grammer or unRestricted Grammer if all Producers are of the form $\alpha \to \beta \in (V \cup T)^+$ and $\beta \in (V \cup T)^+$

In this type of Grammer there exist no Restrictions on length of α & β the only Restrictions is that α Cannot be ϵ

From the Grammer You Can Create recursively Enumerable Variable/language Only turing Machine or Can recognise this language

→ Context Sensitive language (Type 1)

A Grammer G = (VTPS) is Said to be Type 1 if all the Producers are of the form $\alpha \to \beta$ where $\alpha \beta \in (V \cup T)^+$ and length of $\beta \geq \alpha$

Linear Bounded FA Can recognise these language Context Sensitive language Can be derived.

→ Context free Grammer CFG (Type 2)

A Grammer (G) = (V T P S) is Said to be Type 2 or Context free Grammer where if $A \rightarrow \alpha$ and $A \in (V \cup T)^*$ A is Single Non-terminal or Variable

Designed the Pushdown Autometa, it can reggoganize the language by this Grammer

→ Regular Grammer (Type 3)

A Grammer (G) = (V T P S) is Said to Type 3 or RE if the Grammer is right/left letters

$A \rightarrow \alpha\beta$ (Right Side Should be Non-Terminal)

$A \rightarrow \beta\alpha$ (Left Side Should be Non-Terminal)

This Can Regonaize By the Finite Autometa (FA)

Derivation

It is a Process of applhying a Sequencial / Sequance of Production rules inorder to obtain an String.

There are 2 Types of derivation

— Left-Most
— Right-Most

→ left-Most

In Each step, left-Most Non-Terminal is to be Expanded

→ Right-Most In Each step, right-Most Non-Terminal is to be Expanded

→ Derivation Tree [Parse Tree]

Diagrammatical representation of derivation is a Parsh Tree

Three rules must be followed during the Construction of the parse tree

→ root node must be a Start Symbol
→ internal node / intermediate nodes are non-terminal
→ leaf Node are Terminated.

Note- An Grammer Can be Constructed Amigous if there exist More than one RHD / LHD ore More than one Parsh tree for the Given String.

→ Simplification of Context free Grammer

(1) Remove the useless Symbol
(2) Remove the unit production
(3) Removal of Null Productions

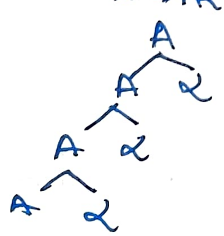
→ These Must be followes To Simplify the Grammer

→ Recursion – Note – top-down approch will not work if the grammer has left Reccursion
– You Must Remove (topdown approoch) left Recursion to apply top-down approch.

Recursion

left Recursion

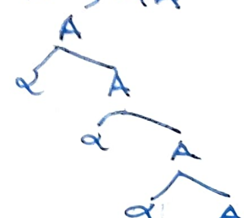
$A \to A\alpha$



$\Rightarrow A\alpha^*$

Right Reccursion

$A \to \alpha A$



$= \alpha^* A$